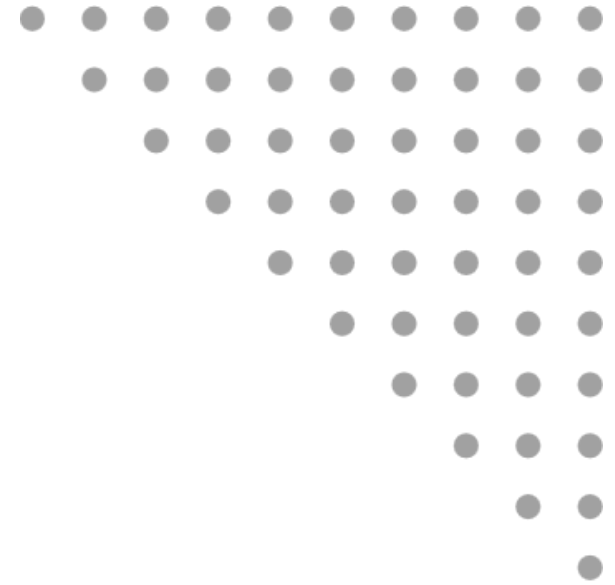




Support Vector Machine (SVM)

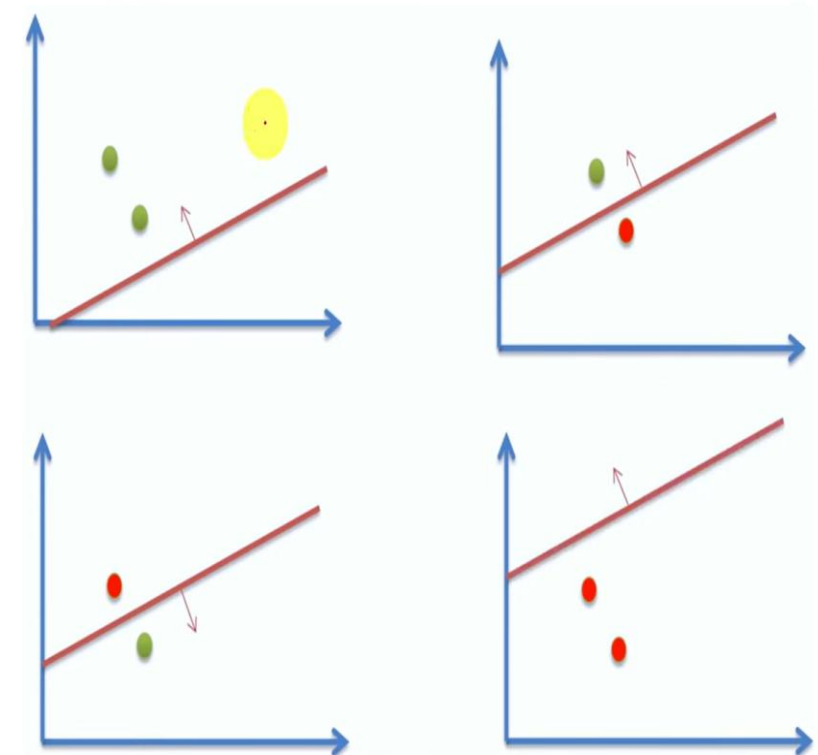
DR. PRASENJIT DEY

Lecture Outline

- Vapnik–Chervonenkis (VC) Theory
- Structural Risk Minimization (SRM)
- Introduction to Support Vector Machines (SVM)
- Working Principle of SVM
- Types of SVM (Hard Margin & Soft Margin)
- Nonlinear Mapping & Feature Space
- Kernel Trick
- Types of Kernel Functions
- Advantages and Limitations of SVM

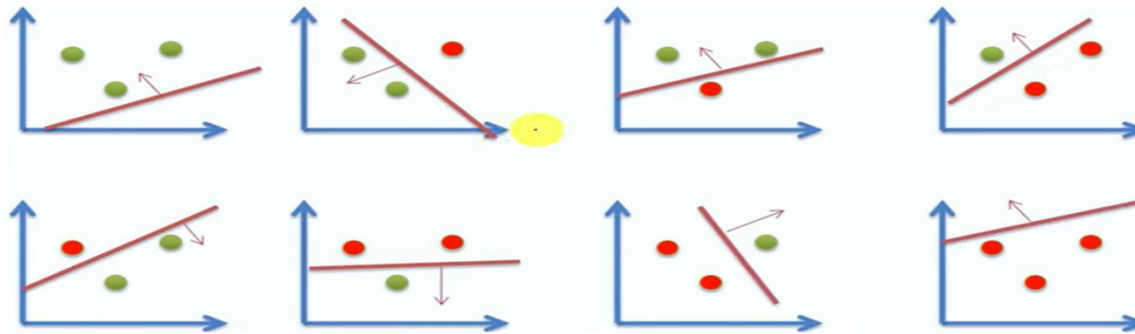
Vapnik-Chervonenkis Theory

- VC theory explains the capacity (complexity) of a hypothesis class used in machine learning to classify data points.
- This capacity is quantified by the VC dimension, defined as:
 - The largest number of points that can be shattered by the model
- Shattering means:
 - The model can correctly realize all possible labelings of those points.
- For n data points, there are 2^n possible labelings (then there are 2^n different ways to classify them).
 - Example:
For 2 points, there are 4 ($= 2^2$) possible labelings
If a model can represent all 4, it shatters 2 points



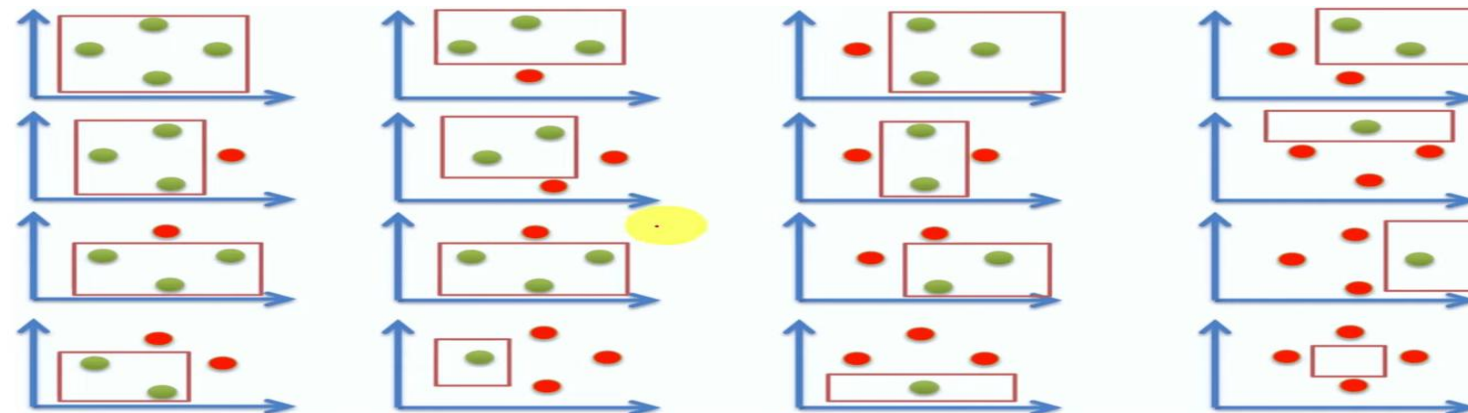
3 Data Points:

A model can shatter 3 points if it can correctly classify them in all 8 possible labelings.



4 Data Points:

- A model can shatter 4 points if it can correctly classify them in all 16 possible labelings.
- A line can't divide them correctly because each side will always have mixed points.
- However, a square boundary can separate them perfectly by enclosing one class inside and keeping the other outside
- The Maximum number of data points that can be shattered by square in 2-Dimensional space is 4



Limitations and Practical Use of VC Theory

- **Complexity:** Understanding and calculating the VC dimension can be mathematically complex.
- **Practical Limits:** A high VC dimension indicates high model capacity, which can lead to overfitting when the training data size is insufficient.
- **Real-World Use:** In practice, we balance VC dimension with other factors like data size and noise.

Practical Use

- **Model Complexity:** A lower VC dimension indicates a less complex model, reducing the risk of overfitting.
- **Training Data:** More training data leads to better generalization by reducing the gap between empirical risk(error rate on the training data) and true risk(error rate on the new/unseen data data).

Structural Risk Minimization (SRM)

- SRM is a principle in ML aimed at finding a balance between a model's complexity and its performance on training data to achieve good generalization on new data.
- **Empirical Risk:** The error rate on the training data
- **Model Complexity:** How complicated the model is. often characterized theoretically by VC dimension (and approximated in practice by model size or regularization)
- **True Risk:** The expected error rate on new, unseen data

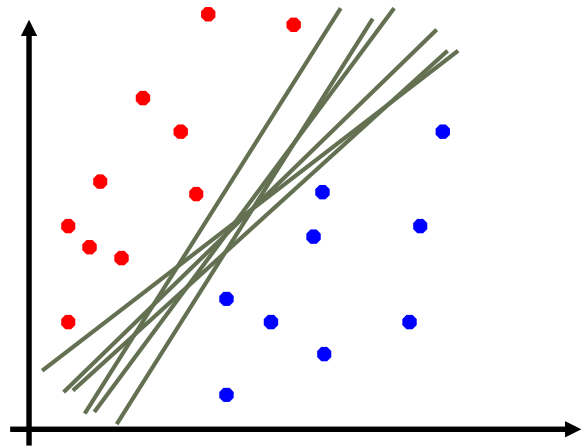
True Risk Estimate $\leq$ Empirical Risk + Complexity Penalty

$$\text{Penalty} \propto \sqrt{\frac{h \left(\log \frac{2N}{h} + 1 \right) - \log \frac{\delta}{4}}{N}}$$

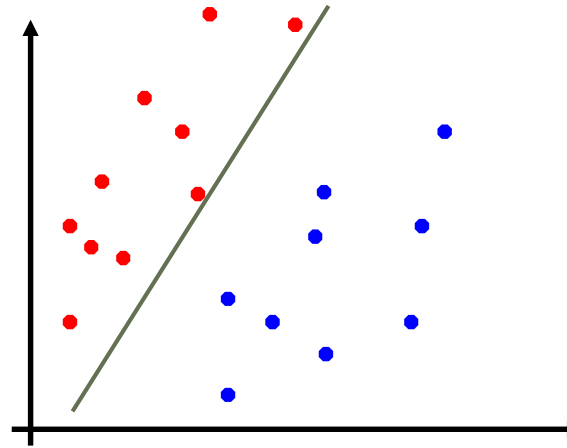
Support Vector Machines

- [Supervised machine learning](#) algorithm used for both classification and regression
- The objective of the SVM algorithm is to find an optimal [hyperplane](#) in an n-dimensional space that can separate the data points in different classes in the feature space.
- The hyperplane tries aims to maximize the margin between the closest points of different classes.
- The dimension of the hyperplane depends upon the number of features.
- SVM is SRM made practical by replacing VC dimension with margin

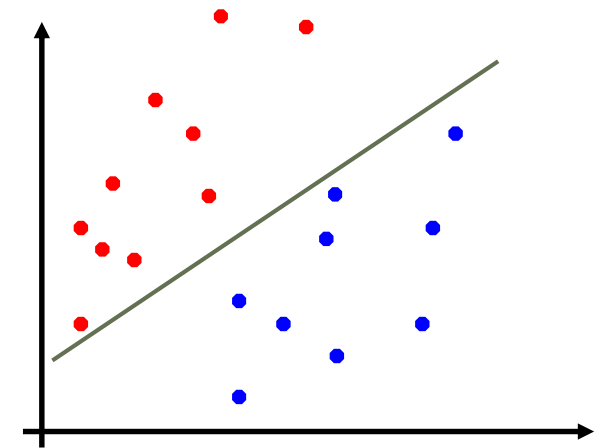
Linear Separators



Multiple decision boundaries can be drawn



Bad decision boundary

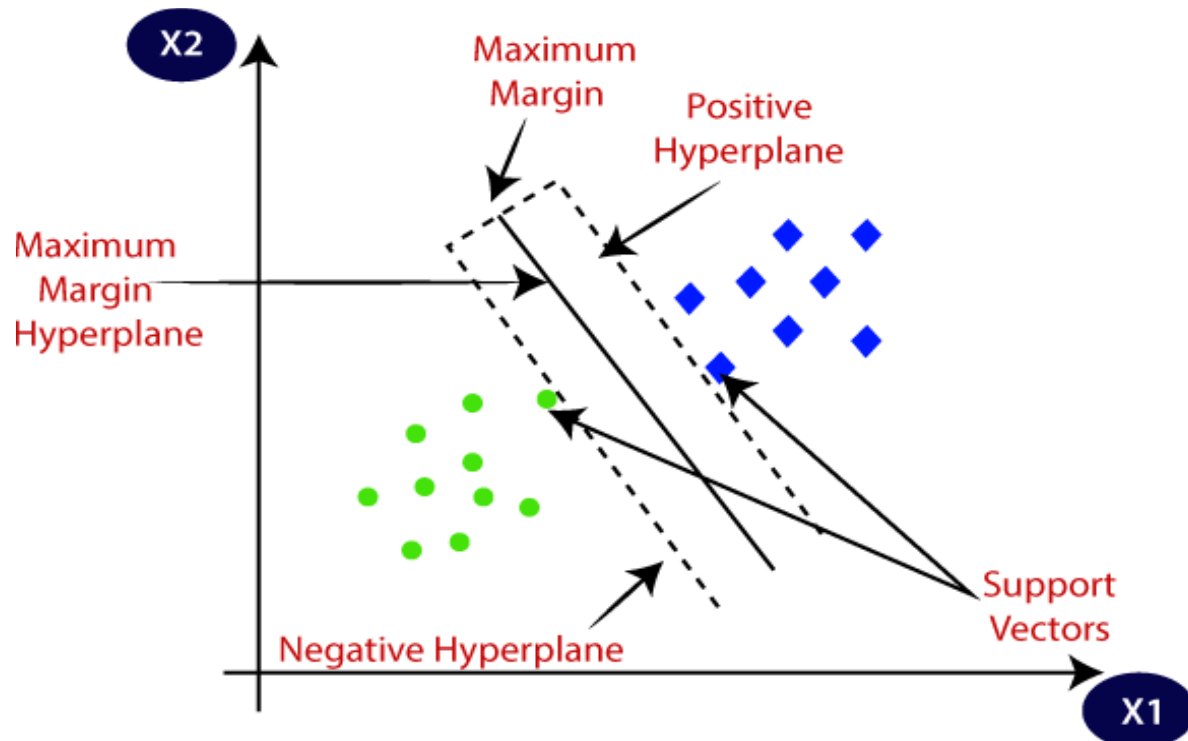


Support Vector Machine (SVM)

SVM is an **approximate** implementation of SRM

Maximizing distance of the decision boundary from training points minimizes the VC dimension resulting into the Good generalization.

Target is Maximizing distance between decision boundary and the training points

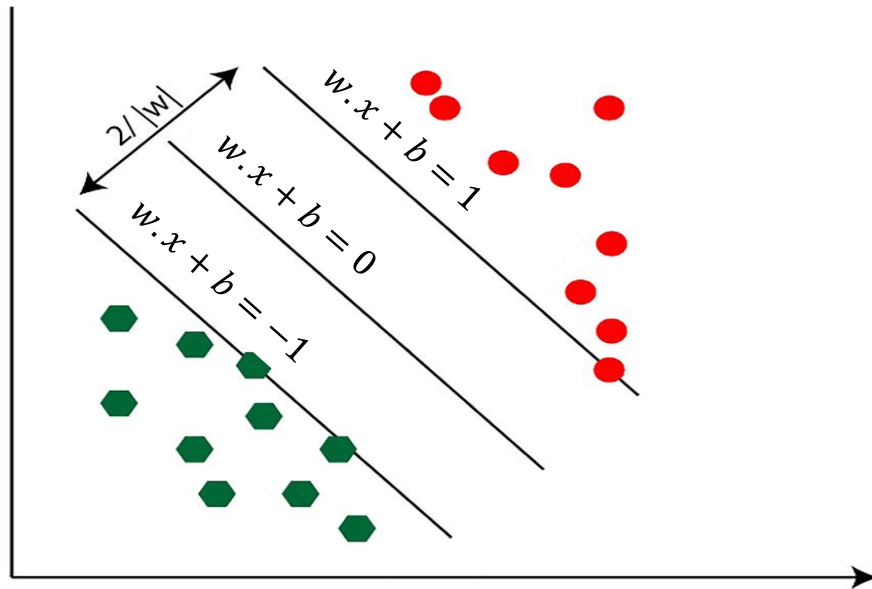


- The margin is the distance between the hyperplane and the closest points from either class. SVM aims to maximize this margin, leading to a more robust classifier.
- Support Vectors: These are the data points that lie closest to the hyperplane. The position of these points is critical as they directly influence the position and orientation of the hyperplane.

SVM

- It separates data into two classes: +1 (positive) and -1 (negative). It uses a linear boundary, or hyperplane, to divide the classes.
- The goal is to find a hyperplane that maximizes the margin between the closest points of each class. This is called a maximum margin classifier or hard margin SVM.

Let, data set $\{x_i, y_i\}; i = 1, \dots, n; \quad x_i \in \mathbb{R}^d \quad \text{and} \quad y_i \in \{-1, +1\}$



separating hyperplane

$$\mathbf{w}^T \mathbf{x} + b = 0$$

s.t.

$$\mathbf{w}^T \mathbf{x}_i + b > 0 \quad \text{if } y_i = +1$$

$$\mathbf{w}^T \mathbf{x}_i + b < 0 \quad \text{if } y_i = -1$$

Suppose training data satisfy following constraints also,

$$\begin{aligned} \mathbf{w}^T \mathbf{x}_i + b &\geq +1 \quad \text{for } y_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b &\leq -1 \quad \text{for } y_i = -1 \end{aligned}$$

Combining these to the one,

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \text{for } \forall i$$

The objective is to find Hyperplane $(\mathbf{w}, b)$ with maximal separation between it and closest data points while satisfying the above constraints

The Maximum Margin Hyperplane

The perpendicular distance d of a data point x from the margin is given by:

$$d = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|} \qquad \|\mathbf{w}\| = \sqrt{\mathbf{w}^T \mathbf{w}}$$

To maximize this distance, we can minimize the square of the denominator to give us a quadratic programming problem given by:

$$\begin{aligned} &\text{Min } \frac{1}{2} \|\mathbf{w}\|^2 \\ &\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \forall i \end{aligned}$$

Solution Via The Method Of Lagrange Multipliers

To solve the above quadratic programming problem with inequality constraints, we can use the method of Lagrange multipliers $L(\mathbf{w}, b, \alpha)$

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1) \rightarrow eq(1)$$

where $\alpha_i = \text{Lagrange Multiplier}$

Now minimizing eq(1) w.r.t. $\mathbf{w}$ and b :

We set derivatives of Lagrangian w.r.t. $\mathbf{w}$ and b to zero $\frac{\partial L}{\partial \mathbf{w}} = 0 \rightarrow eq(2)$, $\frac{\partial L}{\partial b} = 0 \rightarrow eq(3)$

Setting derivative w.r.t. b to zero, it gives:

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n \alpha_i y_i$$

Putting eq(2) we get-

$$\sum_{i=1}^n \alpha_i y_i = 0 \rightarrow eq(4)$$

Setting derivative w.r.t. $\mathbf{w}$ to zero, it gives:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

Putting eq(3) we get-

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \rightarrow eq(5)$$

Solution Via The Method Of Lagrange Multipliers

Expanding Eq(1) we get-

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} (\mathbf{w}^T \mathbf{w}) - \sum_{i=1}^n \alpha_i y_i (\mathbf{w} \cdot \mathbf{x}_i) - \sum_{i=1}^n \alpha_i y_i b + \sum_{i=1}^n \alpha_i \rightarrow eq(6)$$

Putting eq(5) in eq(6)

$$L(\mathbf{w}, b, \alpha) = \frac{1}{2} \sum_{i,j=1}^n \alpha_i y_i \mathbf{x}_i \alpha_j y_j \mathbf{x}_j - \sum_{i=1}^n \alpha_i y_i (\mathbf{w} \cdot \mathbf{x}_i) - \sum_{i=1}^n \alpha_i y_i b + \sum_{i=1}^n \alpha_i \rightarrow eq(7)$$

Putting eq(5) in eq(7)

$$L(b, \alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - b \sum_{i=1}^n \alpha_i y_i \rightarrow eq(8)$$

Putting eq(4) in eq(7)

$$L(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j)$$

Optimized Values of W and b

From, KKT condition

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$$

$$(\mathbf{w}^T \mathbf{x}_i + b) = \frac{1}{y_i}$$

$$b = \frac{1}{y_i} - \mathbf{w}^T \mathbf{x}_i$$

As $y_i = \pm 1$;

$$b = y_i - \mathbf{w}^T \mathbf{x}_i; \quad \mathbf{w} = \sum_{j=1}^n \alpha_j y_j \mathbf{x}_j$$

$$b = y_i - \sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \cdot \mathbf{x}_i$$

For positive support vectors: $\mathbf{w} \cdot \mathbf{x}_i + b = +1 \Rightarrow \mathbf{w} \cdot \mathbf{x}_i = 1 - b$

For negative support vectors: $\mathbf{w} \cdot \mathbf{x}_i + b = -1 \Rightarrow \mathbf{w} \cdot \mathbf{x}_i = -1 - b$

Taking midpoint

$$b = -\frac{\min(\sum_{i:y_i=+1} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i)) + \max(\sum_{i:y_i=-1} \alpha_j y_j (\mathbf{x}_j \cdot \mathbf{x}_i))}{2}$$

$$b = -\frac{1}{2} (\min_{i:y_i=+1} (\mathbf{w} \cdot \mathbf{x}_i) + \max_{i:y_i=-1} (\mathbf{w} \cdot \mathbf{x}_i))$$

The classification of any test point $\mathbf{z}$ can be determined using this expression

$$y(\mathbf{z}) = \text{Sign}(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \cdot \mathbf{z} + b)$$

A positive value of $y(\mathbf{z})$ implies $\mathbf{z} \in +1$ and a negative value means $\mathbf{z} \in -1$

The Dual Problem

The new objective function is in terms of α_i only

It is known as the dual problem: if we know all α_i , we can recover $\mathbf{w}$

The original problem is known as the primal problem

The objective function of the dual problem needs to be maximized (comes out from the KKT theory)

The dual problem is therefore:

$$\max. \mathbf{L}(\boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j)$$

subject to $\alpha_i \geq 0$,

$$\sum_{i=1}^n \alpha_i y_i = 0$$

Properties of α_i when we introduce the Lagrange multipliers

The result when we differentiate the original Lagrangian w.r.t. $\mathbf{b}$

The Dual Problem

$$\begin{aligned} \max. L(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j) \\ \text{subject to } \alpha_i &\geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

This is a quadratic programming (QP) problem

- A global optimum of α always exists and can be found

$\mathbf{w}$ can be recovered by

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

Characteristics of the Solution

Many of the α_i are zero

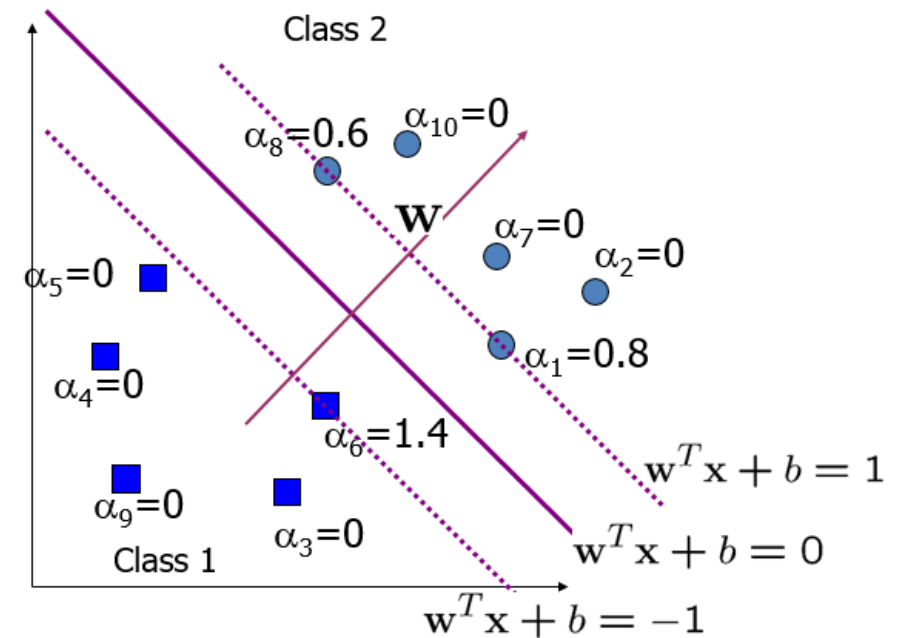
- $\mathbf{w}$ is a linear combination of a small number of data points
- This “sparse” representation can be viewed as data compression as in the construction of kNN classifier

$\mathbf{x}_i$ with non-zero α_i are called support vectors (SV)

- The decision boundary is determined only by the SV
- Let, $(j=1, \dots, s)$ be the indices of the s support vectors.

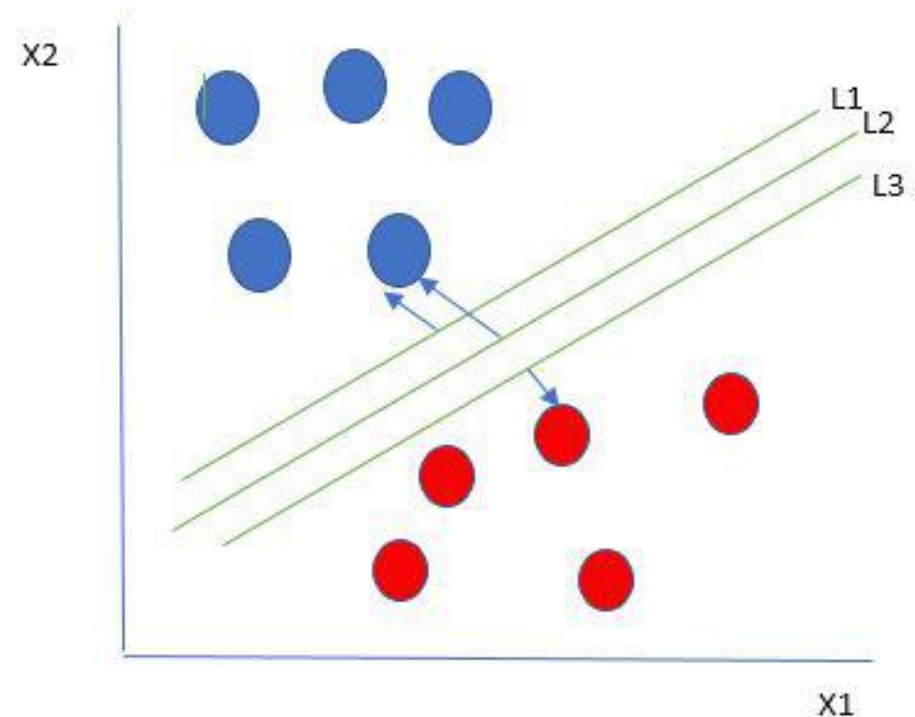
$$\mathbf{w} = \sum_{j=1}^s \alpha_j y_j \mathbf{x}_j$$

- Note: $\mathbf{w}$ need not be formed explicitly



How does SVM work?

- Which line will classify the unseen data well?
 - Select the hyperplane that maximizes the margin between itself and the nearest data points from each class



How to Select Best Hyperplane

The goal is to choose the hyperplane with the largest margin

Here's how it works with the given example:

Hyperplane for classifier 1 = $5 + 2x_1 + 5x_2$

Hyperplane for classifier 2 = $5 + 20x_1 + 50x_2$

1. Calculate the Weight Vector:

1. For the first hyperplane: The weight vector is (2,5)
2. For the second hyperplane: The weight vector is (20,50)

2. Compute the Distance Error

Distance error function of SVM is given by: $\sqrt{\mathbf{w}_1^2 + \mathbf{w}_2^2}$

- The weight vector for the first equation omitting the intercept is (2, 5)
- Distance Error 1 = $\sqrt{2^2 + 5^2} = 5.39$
- The weight vector for the first equation omitting the intercept is (20, 50)

Distance Error 2 = $\sqrt{20^2 + 50^2} = 53.85$

Calculate the Margin:

- The margin is inversely proportional to the norm of the weight vector. It is calculated as $2/\|\mathbf{w}\|$.

$$\text{For Classifier 1} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{5.39} = 0.37$$

$$\text{For Classifier 2} = \frac{2}{\|\mathbf{w}\|} = \frac{2}{53.85} = 0.037$$

Select the Hyperplane with the Larger Margin:

1. Compare the margins:

Classifier 1: Margin ≈ 0.37

Classifier 2: Margin ≈ 0.037

2. Classifier 1 has a larger margin

- Classifier 1** is selected because it has a larger margin (0.37) compared to Classifier 2 (0.037)
- A larger margin means better separation between classes and better generalization to new data

SVM Classifier Algorithm

- Given a two-class linearly separable dataset of n points of the form

$$(\vec{x}_1, y_1), (\vec{x}_2, y_2) \dots (\vec{x}_n, y_n)$$

where the y_i 's are either +1 or -1

- Find $\vec{\alpha} = \alpha_1, \alpha_2 \dots \alpha_n$ which maximizes

$$\phi(\vec{\alpha}) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\vec{x}_i \cdot \vec{x}_j)$$

- Subject to, $\alpha_i \geq 0, \sum_{i=1}^n \alpha_i y_i = 0$

- Compute,

$$\vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i$$

- Compute,

$$b = \frac{1}{2} (\min_{i: y = +1} (\vec{w} \cdot \vec{x}_i) + \max_{i: y = -1} (\vec{w} \cdot \vec{x}_i))$$

- The SVM classifier function is given by

$$F(\vec{x}) = \vec{w} \cdot \vec{x} + b$$

Linear Example – when Φ is trivial

Suppose we are given the following positively labeled data points in $\mathbb{R}^2$:

$$\left\{ \begin{pmatrix} 3 \\ 1 \end{pmatrix}, \begin{pmatrix} 3 \\ -1 \end{pmatrix}, \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \begin{pmatrix} 6 \\ -1 \end{pmatrix} \right\}$$

and the following negatively labeled data points in $\mathbb{R}^2$ (see Figure 1):

$$\left\{ \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ -1 \end{pmatrix}, \begin{pmatrix} -1 \\ 0 \end{pmatrix} \right\}$$

$$\left\{ s_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, s_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}, s_3 = \begin{pmatrix} 3 \\ -1 \end{pmatrix} \right\}$$

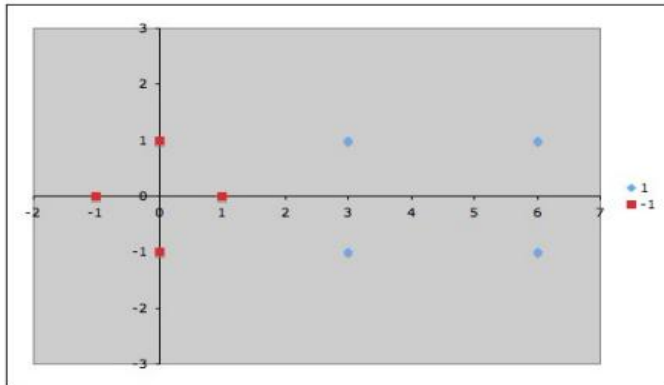


Figure 1: Sample data points in $\mathbb{R}^2$. Blue diamonds are positive examples and red squares are negative examples.

In what follows we will use vectors augmented with a 1 as a bias input, and for clarity we will differentiate these with an over-tilde. So, if $s_1 = (10)$, then $\tilde{s}_1 = (101)$. Figure 3 shows the SVM architecture, and our task is to find values for the α_i such that

$$\begin{aligned} \alpha_1 \Phi(s_1) \cdot \Phi(s_1) + \alpha_2 \Phi(s_2) \cdot \Phi(s_1) + \alpha_3 \Phi(s_3) \cdot \Phi(s_1) &= -1 \\ \alpha_1 \Phi(s_1) \cdot \Phi(s_2) + \alpha_2 \Phi(s_2) \cdot \Phi(s_2) + \alpha_3 \Phi(s_3) \cdot \Phi(s_2) &= +1 \\ \alpha_1 \Phi(s_1) \cdot \Phi(s_3) + \alpha_2 \Phi(s_2) \cdot \Phi(s_3) + \alpha_3 \Phi(s_3) \cdot \Phi(s_3) &= +1 \end{aligned}$$

Since for now we have let $\Phi() = I$, this reduces to

$$\begin{aligned} \alpha_1 \tilde{s}_1 \cdot \tilde{s}_1 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_1 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_1 &= -1 \\ \alpha_1 \tilde{s}_1 \cdot \tilde{s}_2 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_2 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_2 &= +1 \\ \alpha_1 \tilde{s}_1 \cdot \tilde{s}_3 + \alpha_2 \tilde{s}_2 \cdot \tilde{s}_3 + \alpha_3 \tilde{s}_3 \cdot \tilde{s}_3 &= +1 \end{aligned}$$

Now, computing the dot products:

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = -1$$

$$2 \alpha_1 + 4 \alpha_2 + 4 \alpha_3 = -1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} = +1$$

$$4 \alpha_1 + 11 \alpha_2 + 9 \alpha_3 = +1$$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \cdot \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} = +1$$

$$4 \alpha_1 + 9 \alpha_2 + 11 \alpha_3 = +1$$

$$\begin{aligned} 2\alpha_1 + 4\alpha_2 + 4\alpha_3 &= -1 \\ 4\alpha_1 + 11\alpha_2 + 9\alpha_3 &= +1 \\ 4\alpha_1 + 9\alpha_2 + 11\alpha_3 &= +1 \end{aligned}$$

A little algebra reveals that the solution to this system of equations is $\alpha_1 = -3.5, \alpha_2 = 0.75$ and $\alpha_3 = 0.75$.

$$\begin{aligned} \tilde{w} &= \sum_i \alpha_i \tilde{s}_i \\ &= -3.5 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ 1 \\ 1 \end{pmatrix} + 0.75 \begin{pmatrix} 3 \\ -1 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \\ -2 \end{pmatrix} \end{aligned}$$

Finally, remembering that our vectors are augmented with a bias, we can equate the last entry in $\tilde{w}$ as the hyperplane offset b and write the separating hyperplane equation $y = wx + b$ with $w = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and $b = -2$. Plotting the line gives the expected decision surface (see Figure 4).

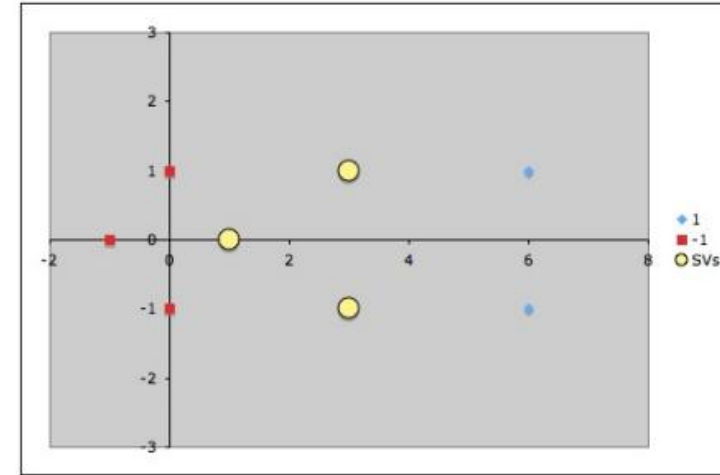


Figure 2: The three support vectors are marked as yellow circles.

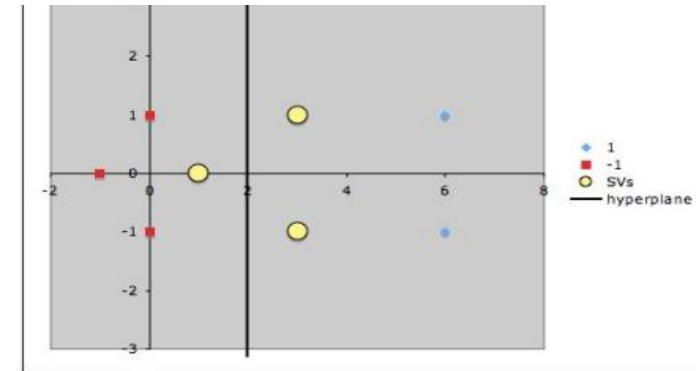
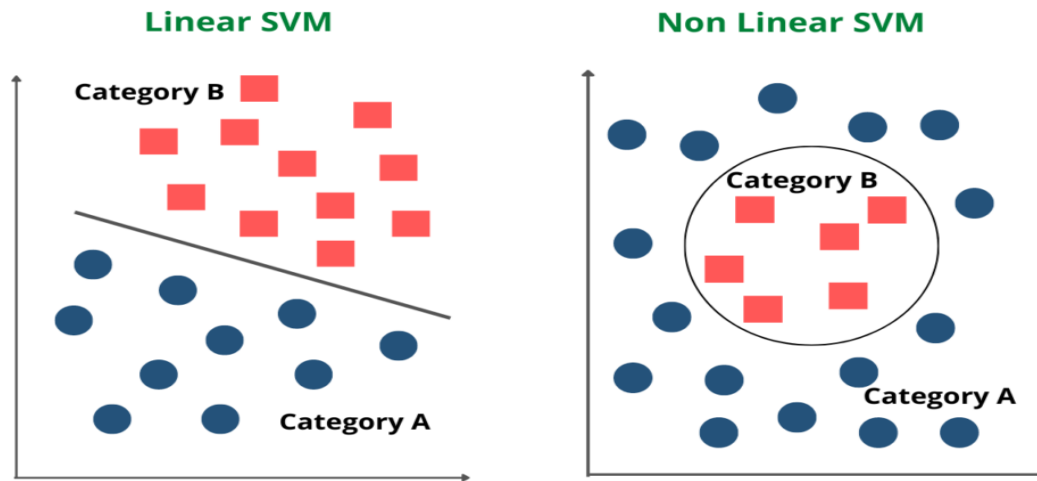


Figure 4: The discriminating hyperplane corresponding to the values $\alpha_1 = -3.5, \alpha_2 = 0.75$ and $\alpha_3 = 0.75$.

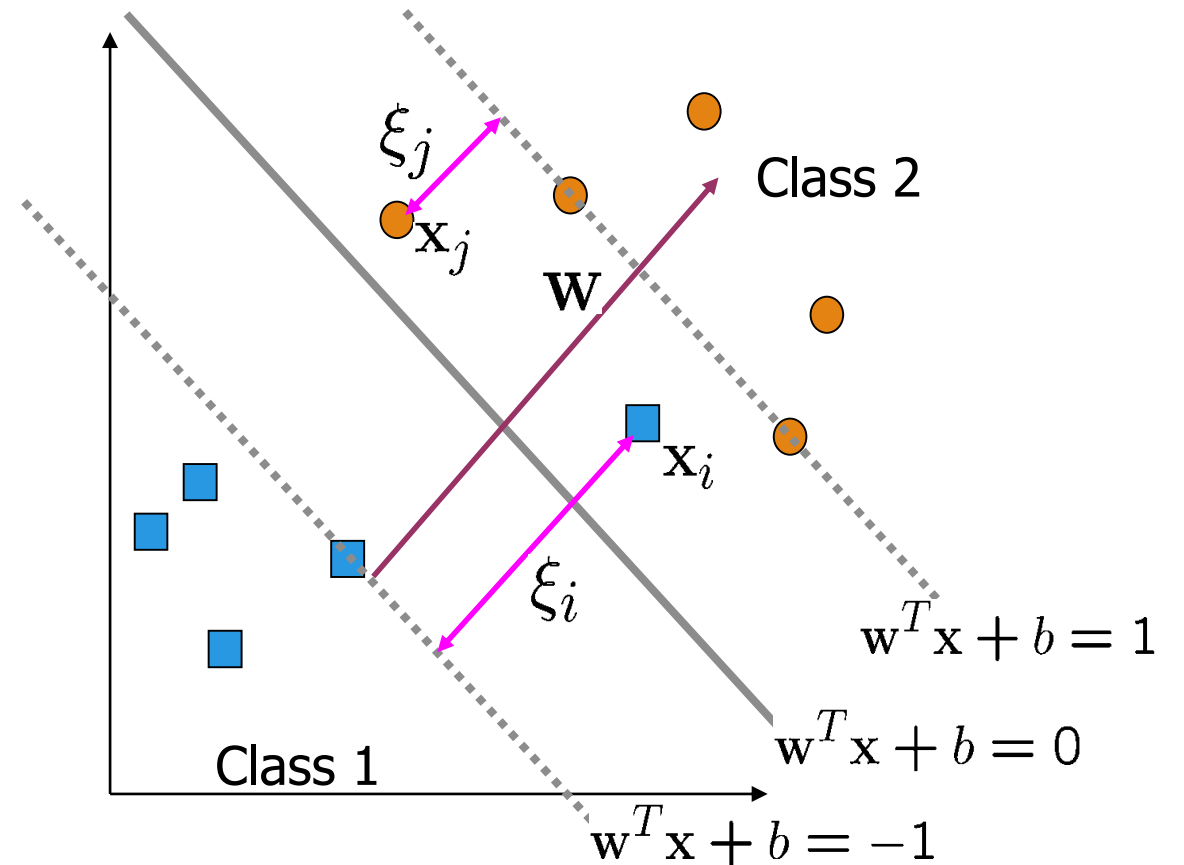
Types of SVM

- **Linear SVM:** Linear SVM is used for linearly separable data, which means if a dataset can be classified into two classes by using a Hyperplane.
- **Non-linear SVM:** Non-Linear SVM is used for non-linearly separated data, which means if a dataset cannot be classified by using a Hyperplane.



Non-linearly Separable Problems

- We allow “error” x_i in classification
- it is based on the output of the discriminant function $w^T x + b$
- x_i approximates the number of misclassified samples



Soft Margin Hyperplane

The new conditions become

$$\begin{cases} (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i & y_i = 1 \\ (\mathbf{w}^T \mathbf{x}_i + b) \leq -1 + \xi_i & y_i = -1 \\ \xi_i \geq 0 & \forall_i \end{cases}$$

- ξ_i are “slack variables” in optimization
- Note that $\xi_i=0$ if there is no error for $\mathbf{x}_i$
- ξ_i is an upper bound of the number of errors

We want to minimize $\frac{1}{2} \|\vec{w}\|^2 + C \sum_{i=1}^n \xi_i$ subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \xi_i \geq 0$

C : tradeoff parameter between error and margin

The Optimization Problem

With α and μ Lagrange multipliers, POSITIVE

$$L = \frac{1}{2} \vec{w}^T \vec{w} + C \sum_{i=1}^n \xi_i + \sum_{i=1}^n \alpha_i (1 - \xi_i - y_i (\vec{w}^T \vec{x}_i + b)) - \sum_{i=1}^n \mu_i \xi_i$$

$$\frac{\partial L}{\partial \vec{w}} = \vec{w} - \sum_{i=1}^n \alpha_i y_i \vec{x}_i = 0$$

$$\vec{w} = \sum_{i=1}^n \alpha_i y_i \vec{x}_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0$$

$$\frac{\partial L}{\partial b} = \sum_{i=1}^n \alpha_i y_i = 0$$

The Dual Problem

$$L = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \vec{x}_i^T \vec{x}_j + C \sum_{i=1}^n \xi_i + \\ + \sum_{i=1}^n \alpha_i \left(1 - \xi_i - y_i \left(\sum_{j=1}^n \alpha_j y_j \vec{x}_j^T \vec{x}_i + b \right) \right) - \sum_{i=1}^n \mu_i \xi_i$$

$$\text{With } \sum_{i=1}^n y_i \alpha_i = 0 \qquad C = \alpha_i + \mu_i$$

$$L = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \vec{x}_i^T \vec{x}_j + \sum_{i=1}^n \alpha_i$$

The Optimization Problem

The dual of this new constrained optimization problem is

$$\begin{aligned} \max. \mathbf{w}(\alpha) &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j) \\ \text{subject to } c &\geq \alpha_i \geq 0, \quad \sum_{i=1}^n \alpha_i y_i = 0 \\ C &= \alpha_i + \mu_i \end{aligned}$$

New constraints derive from since μ and α are positive.

$\mathbf{w}$ is recovered as $\mathbf{w} = \sum_{j=1}^{sv} \alpha_j y_j \mathbf{x}_j$, where sv is the numbers or support vector ($1 \leq sv \leq n$)

This is very similar to the optimization problem in the linear separable case, except that there is an upper bound C on α_i now

Once again, a QP solver can be used to find α_i

The Optimization Problem

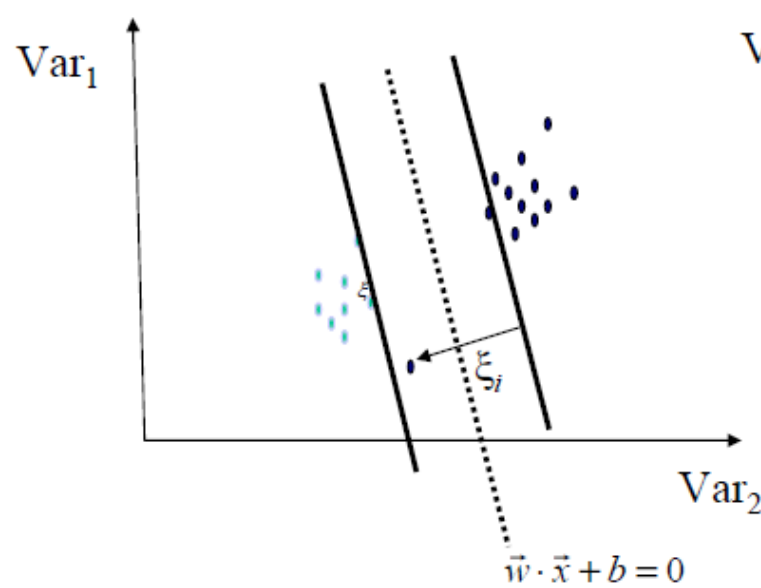
$$\frac{1}{2} \|\vec{w}\|^2 + C \sum_{i=1}^n \xi_i$$

The algorithm try to keep ξ_i null, maximizing the margin

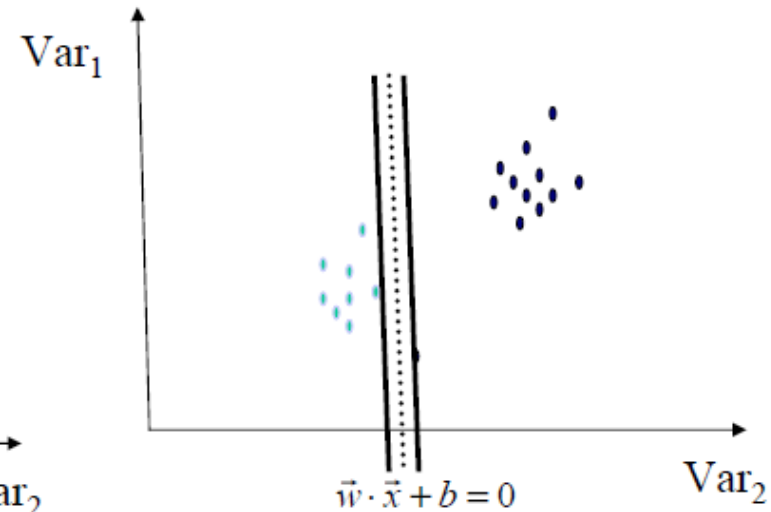
The algorithm does not minimize the number of error. Instead, it minimizes the sum of distances from the hyperplane

When C increases the number of errors tend to lower. At the limit of C tending to infinite, the solution tend to that given by the hard margin formulation, with 0 errors

Soft margin is more robust

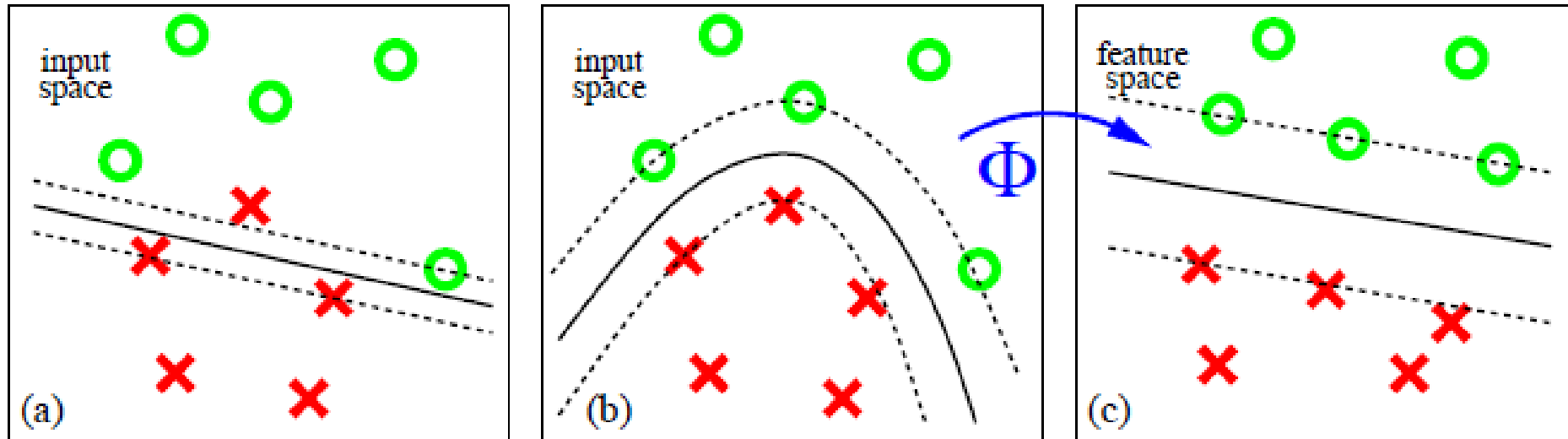


Soft Margin SVM



Hard Margin SVM

Find a feature space

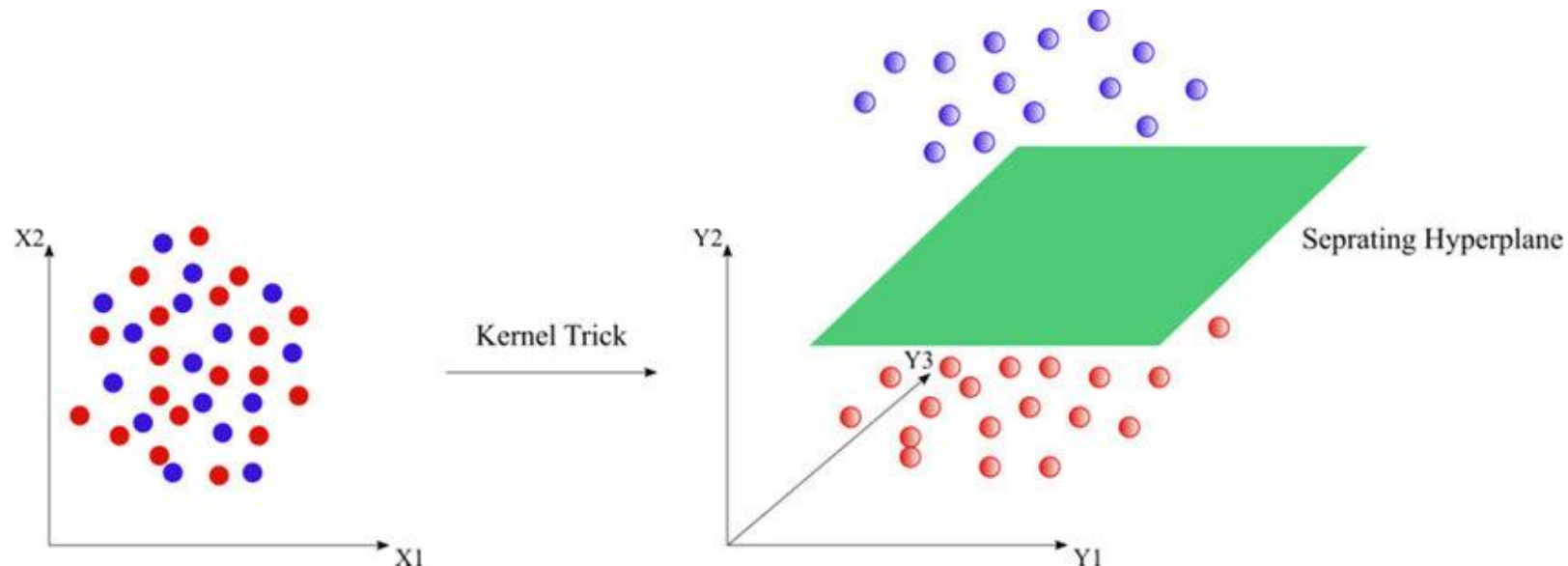


What if data is nonlinearly separable?

The maximal margin hyperplane can classify only linearly separable data

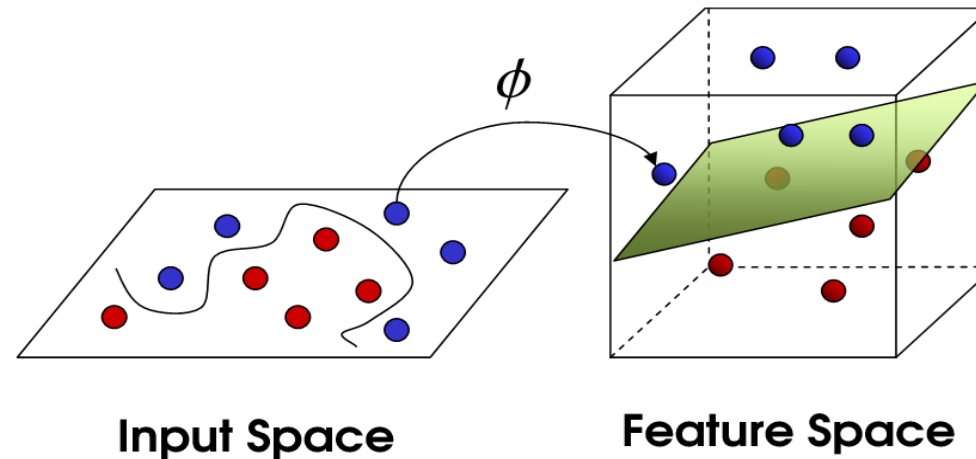
What if the data is linearly non-separable?

Transform your data into a higher-dimensional space to make it linearly separable, and then use a hyperplane that maximizes the margin



Non-linear data mapping

- For non-linear data in SVM, we utilize a mapping function to transform the input space into a higher-dimensional feature space.
- This transformation allows the SVM to identify a hyperplane that can effectively separate the data points in this new space, thereby enabling accurate classification despite the original data exhibiting complex, non-linear relationships
- For example, one mapping function $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$ used to transform a 2D data to 3D data is given as follows:
$$\phi(x, y) = (x^2, \sqrt{2}xy, y^2)$$
- Consider a point (2, 3) in 2D space, convert it into 3D space
- Here $x = 2$ and $y = 3$,
- Hence point in 3D space is:
 $(2^2, \sqrt{2}(3*2), 3^2) = (4, 6\sqrt{2}, 9)$



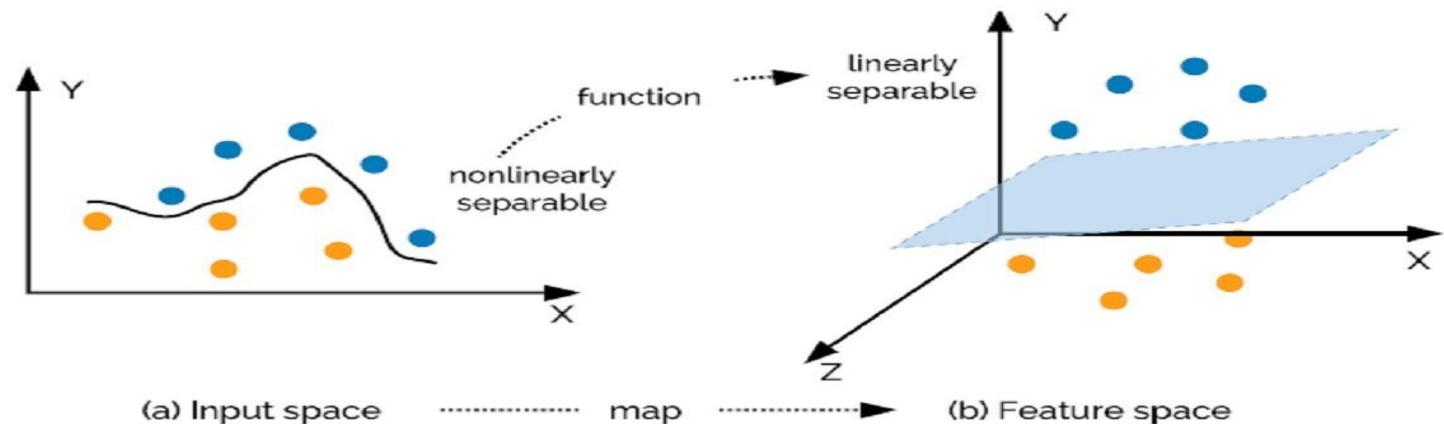
Mapping Functions and Challenges

- Mapping involves more computations and learning costs.
- No generalized rule for transformations.
- Large data can slow down mapping process.
- High computational costs due to polynomial combinations in complex data.

Kernels are useful for computing values without transforming data.

What is a Kernel?

Kernels are a set of functions used to transform data from lower dimension to higher dimension and to manipulate data using dot product at higher dimensions.



Kernel Function

- Kernel function accepts inputs in the original lower dimensional space and returns the dot product of the transformed vectors in the higher dimensional space.
- It can somewhat help to understand how the kernel function is equal to the dot product of the transformed vectors by considering that each coordinate of the transformed vector $\phi(x)$ is just some function of the coordinates in the corresponding lower dimensional vector x .

e.g., the kernel trick for the 2nd-degree polynomial is illustrated below

Equation 5-9. Kernel trick for a 2nd-degree polynomial mapping

$$\begin{aligned}\phi(\mathbf{a})^T \cdot \phi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^T \cdot \mathbf{b})^2\end{aligned}$$

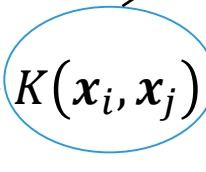
Modification Due to Kernel Function

Change all inner products to kernel functions

Original
$$\max. \mathbf{w}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j)$$

subject to $c \geq \alpha_i \geq 0$, $\sum_{i=1}^n \alpha_i y_i = 0$, $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$

With kernel function
$$\max. \mathbf{w}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$



subject to $c \geq \alpha_i \geq 0$, $\sum_{i=1}^n \alpha_i y_i = 0$

Why do we need Kernel Trick

Kernel trick means replacing the dot product in mapping functions with a kernel function.

$$k(x, y) = \phi(x) \cdot \phi(y)$$

- Similar to mapping functions, kernels help in mapping data from input space to higher-dimensional feature space with least computations.
- Performing the kernel operation is much easier compared to mapping functions.

This is illustrated in the following numerical example.

Consider two data points (1, 2) and (3, 4)

Apply a polynomial kernel $k(x, y) = (x^T y)^2$ and show that it is equivalent to mapping function

$$\phi = (x^2, y^2, \sqrt{2} xy)$$

Solution:

$$\phi(1, 2) = (1^2, 2^2, \sqrt{2} (1 * 2)) = (1, 4, 2\sqrt{2})$$

$$\phi(3, 4) = (3^2, 4^2, \sqrt{2} (3 * 4)) = (9, 16, 12\sqrt{2})$$

$$\phi(1, 2) \cdot \phi(3, 4) = (1, 4, 2\sqrt{2}) \cdot (9, 16, 12\sqrt{2}) = (1 \times 9 + 4 \times 16 + 24(2)) = 121$$

Now, using polynomial kernel function, $k(x, y) = (x^T y)^2$, it can be computed as:

$$\left(\begin{pmatrix} 1 \\ 2 \end{pmatrix} \cdot \begin{pmatrix} 3 & 4 \end{pmatrix} \right)^2 = 11^2 = 121$$

It can be seen the computation involves many multiplication operations.

The operation can be computed quickly using kernel functions.

EXAMPLE

Suppose we have 5 1D data points

$$x_1 = 1, x_2 = 2, x_3 = 4, x_4 = 5, x_5 = 6,$$

1, 2, 6 as class 1 and 4, 5 as class 2

$$\Rightarrow y_1 = 1, y_2 = 1, y_3 = -1, y_4 = -1, y_5 = 1$$

- We use the polynomial kernel of degree 2

$$K(x, y) = (xy + 1)^2$$

C is set to 100

- We first find α_i ($i=1, \dots, 5$) by

$$\max. \sum_{i=1}^5 \alpha_i - \frac{1}{2} \sum_{i,j=1}^5 \alpha_i \alpha_j y_i y_j (x_i x_j + 1)^2$$

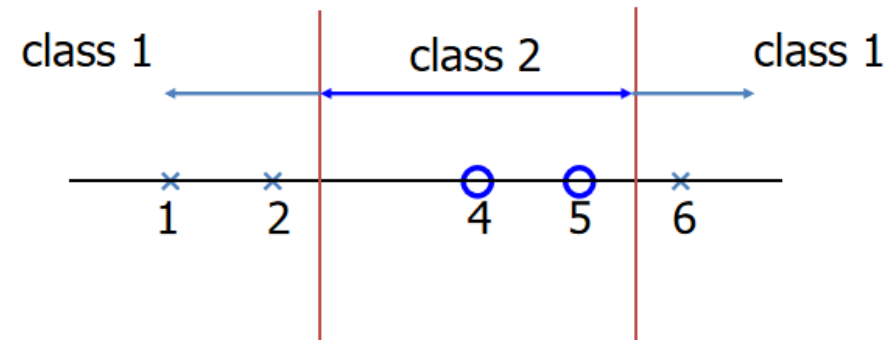
$$\text{Subject to } 100 \geq \alpha_i \geq 0, \quad \sum_{i=1}^5 \alpha_i y_i = 0$$

- By using a QP solver, we get

$$\alpha_1 = 0, \alpha_2 = 2.5, \alpha_3 = 0, \alpha_4 = 7.333, \alpha_5 = 4.833$$

Note that the constraints are indeed satisfied

The support vectors are $\{x_2 = 2, x_4 = 5, x_5 = 6\}$



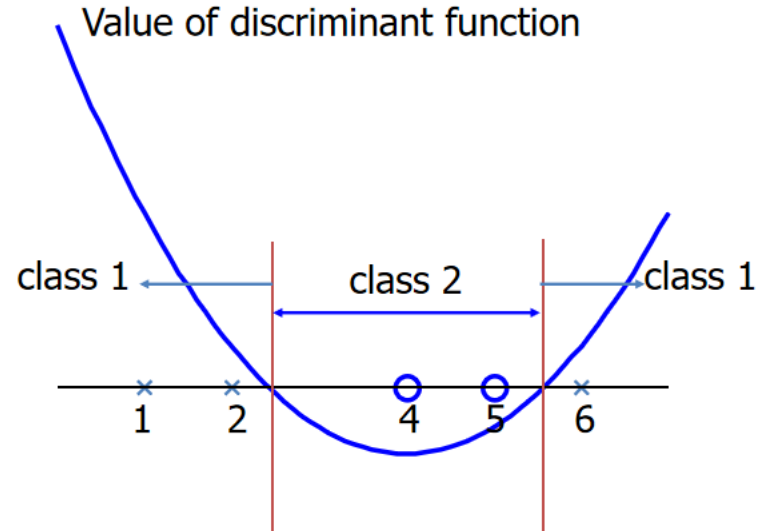
- The discriminant function is

$$\begin{aligned}
 f(z) &= 2.5(1)(2z+1)^2 + 7.333(-1)(5z+1)^2 + 4.833(1)(6z+1)^2 + b \\
 &= 0.6667z^2 - 5.333z + b
 \end{aligned}$$

α_5 y_5 $K(z, x_5)$

- b is recovered by solving $f(2)=1$ or by $f(5)=-1$ or by $f(6)=1$,
All three give $b=9$

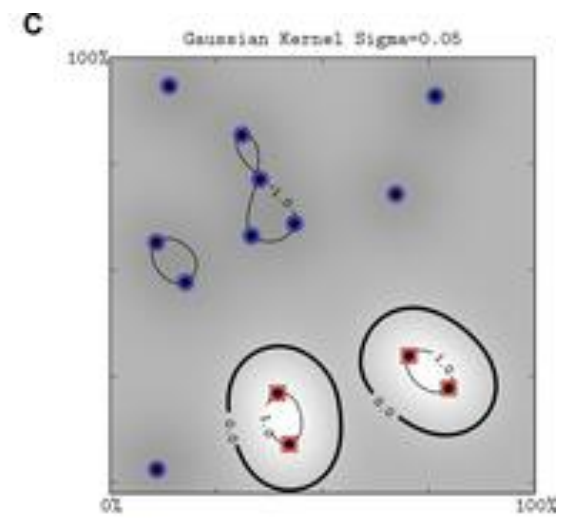
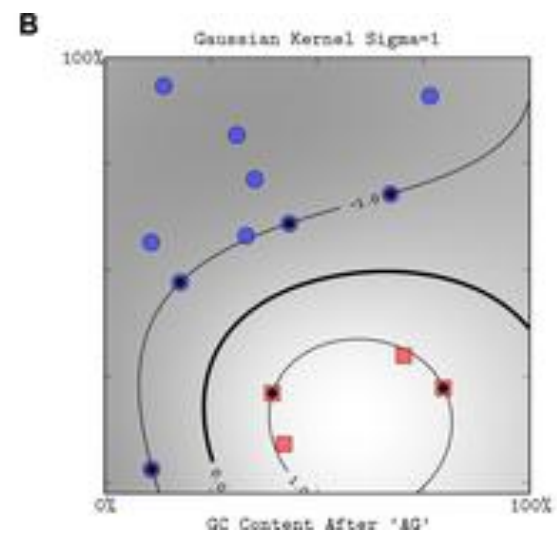
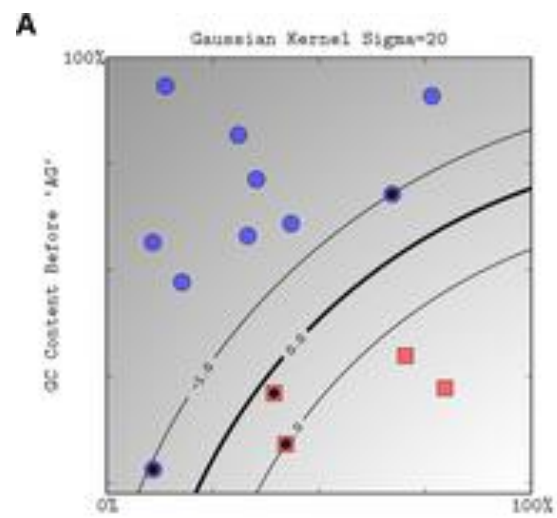
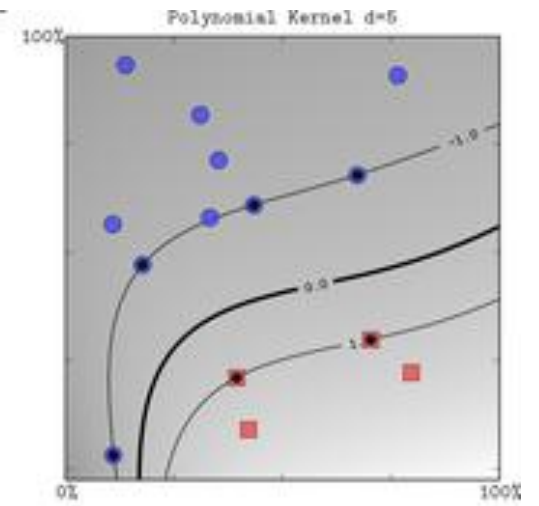
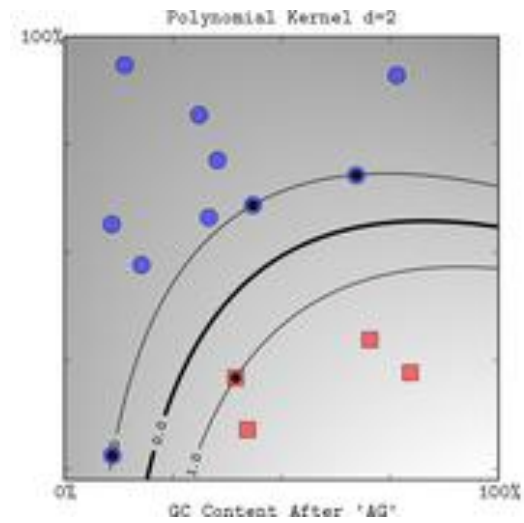
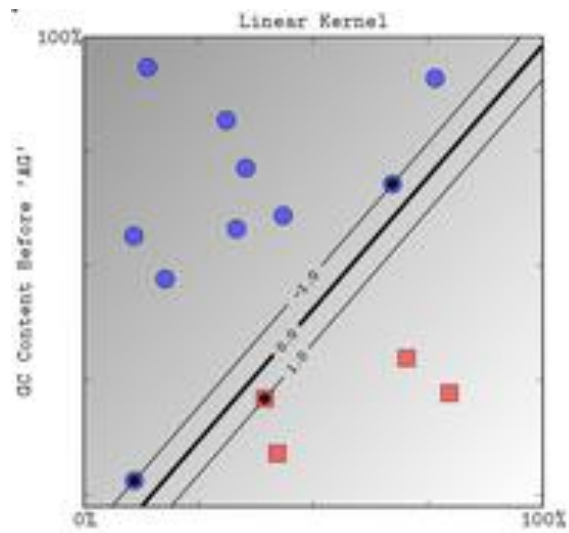
$$f(z) = 0.6667z^2 - 5.333z + 9$$



Common Kernel Functions

Some commonly used kernel functions:

- Linear: $K(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$
- Polynomial of degree d : $K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^d$
- Gaussian Radial Basis Function(RBF): $K(\mathbf{x}, \mathbf{y}) = e^{-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}}$



Choosing the Kernel Function

- One of the most important steps in SVM
- Kernel function maps data into **higher-dimensional space**
- Common types:
 - Linear
 - Polynomial
 - RBF (Gaussian)
- Practical tips:
 - Start with **RBF kernel**
 - Tune parameters (C , σ)
- Kernel choice affects **model performance and complexity**

Advantages

- Effective in high-dimensional spaces: SVMs perform well even with larger features than observations.
- Resistant to overfitting: SVMs use the margin maximization principle to generalize well to unseen data.
- Versatile: SVMs can be applied to both classification and regression problems, supporting different kernel functions.
- Effective in limited data cases: SVMs work well even with small training data sets.
- Ability to handle nonlinear data: SVMs use kernel functions to transform input into a higher-dimensional feature space, enabling linear decision boundaries.

Disadvantages

Computational Intensity:

- SVMs can be computationally expensive, especially with large data sets.
- Training time and memory requirements increase with the number of training samples.

Sensitive Parameter Tuning:

- SVM performance can be sensitive to parameter settings like regularization parameter and kernel function choice.

Lack of Probabilistic Outputs:

- SVMs provide binary classification outputs but do not directly estimate class probabilities.

Difficulty in Interpreting Complex Models:

- SVMs create complex decision boundaries, making it difficult to interpret and understand data patterns.

Scalability Issues:

- SVMs may face scalability issues when applied to large data sets.

THANK YOU